
Universidad Técnica Estatal de Quevedo

Carrera de Ingeniería en Software

GA — Práctica en Clase: Proyecto ASP.NET Core MVC

*Inicialización de proyecto, Modelo con Data Annotations,
Controller, Vistas Razor y Validación en Servidor*

Asignatura: Aplicaciones Web

Código / Paralelo: SOFT-R-A | 5to Nivel

Estudiante: Loor Medranda Marlon Taylor

Período: UTEQ 2026 | Corte 1

Fecha: 18 de junio de 2026

Repositorio: github.com/mloorm14/PCAspCoreMVCp1

Índice

1. Introducción	2
2. Marco Teórico	2
2.1. Patrón MVC y arquitectura del pipeline en ASP.NET Core	2
2.2. Data Annotations: mecanismo interno y jerarquía de atributos	3
2.3. Model Binding: del protocolo HTTP al objeto C#	4
2.4. Validación en servidor frente a validación en cliente	5
2.4.1. Validación en el servidor (<i>server-side</i>)	5
2.4.2. Validación en el cliente (<i>client-side</i>)	5
2.5. Protección CSRF y el mecanismo <i>AntiForgeryToken</i>	6
2.5.1. Naturaleza del ataque CSRF	6
2.5.2. El patrón <i>Synchronizer Token</i> en ASP.NET Core	6
3. Desarrollo e Implementación	7
3.1. Fase 1: Inicialización del proyecto con .NET CLI	7
3.2. Fase 2: Modelo Producto con Data Annotations	9
3.3. Fase 3: ProductoController con acciones GET y POST	11
3.4. Fase 4: Vista Razor Index.cshtml	13
3.5. Fase 5: Vista Razor Create.cshtml con AntiForgeryToken	15
3.6. Fase 6: Configuración del pipeline en Program.cs	18
4. Verificación con Casos de Prueba	19
4.1. Caso 1 — Todos los campos vacíos	19
4.2. Caso 2 — Valores numéricos fuera de rango	20
4.3. Caso 3 — Inspección del token CSRF en DevTools	21
4.4. Caso 4 — Intento de inyección XSS en el campo Nombre	22
4.5. Caso 5 — Datos completamente válidos y flujo exitoso	23
5. Conclusiones	25

1. Introducción

La presente práctica corresponde a la *Unidad II — Herramientas de Programación Web del Servidor* de la asignatura **Aplicaciones Web**. Su objetivo central es construir una aplicación web funcional en **ASP.NET Core MVC** empleando el SDK de .NET 8, profundizando en los mecanismos que gobiernan el procesamiento de formularios del lado del servidor: *Model Binding*, validación mediante *Data Annotations* y protección contra ataques *Cross-Site Request Forgery* (CSRF) con el mecanismo **AntiForgeryToken**.

*“El estudiante inicializará un proyecto ASP.NET Core MVC mediante la CLI de .NET, diseñará un modelo **Producto** con restricciones de datos declarativas, implementará un controlador con acciones **GET/POST**, construirá vistas **Razor** con validación reactiva y documentará el comportamiento del pipeline de validación ante entradas válidas e inválidas.”*

ASP.NET Core es el sucesor multiplataforma del clásico ASP.NET Framework. Publicado inicialmente en 2016 y consolidado en su arquitectura actual a partir de .NET 5 (2020), permite desarrollar aplicaciones web de alto rendimiento sobre Linux, macOS y Windows [1]. El patrón **MVC** (Modelo–Vista–Controlador) que implementa separa las responsabilidades en tres capas ortogonales, facilitando la prueba unitaria, el mantenimiento y la escalabilidad horizontal del sistema [2].

Los entregables de esta práctica son cinco artefactos de software, todos dentro del proyecto **MiPrimerMVC**:

- **Models/Producto.cs** — Modelo C# con *Data Annotations* para validación declarativa.
- **Controllers/ProductoController.cs** — Controlador con acciones **Index** (GET) y **Create** (GET + POST).
- **Views/Producto/Index.cshtml** — Vista **Razor** que lista los productos registrados.
- **Views/Producto/Create.cshtml** — Formulario **Razor** con `asp-antiforgery="true"` y *tag helpers* de validación.
- **Program.cs** — Punto de entrada y configuración del pipeline de middlewares.

2. Marco Teórico

2.1. Patrón MVC y arquitectura del pipeline en ASP.NET Core

El patrón **Modelo–Vista–Controlador** fue descrito por Trygve Reenskaug en Xerox PARC en 1979 y formalizado para el desarrollo web por Ruby on Rails en 2004. En ASP.NET Core, su implementación aprovecha el pipeline de *middlewares* que proporciona **IApplicationBuilder** [3].

Una petición HTTP en ASP.NET Core MVC sigue el siguiente flujo de ejecución (Figura ??):

1. **Kestrel** recibe la petición TCP/IP y la convierte en un objeto **HttpContext**.

2. El *middleware* de **Routing** examina la URL y asocia un *endpoint* mediante la tabla de rutas convencionales (`{controller}/{action}/{id?}`).
3. El *middleware* de **Autorización** verifica permisos (irrelevante en esta práctica, pero parte del pipeline).
4. El **MVC Middleware** instancia el controlador usando el contenedor de *Dependency Injection* (`IServiceProvider`).
5. Se ejecuta la cadena de **filtros de acción** (`IActionFilter`): `OnActionExecuting`, ejecución de la acción, `OnActionExecuted`.
6. El **Model Binder** extrae valores del cuerpo del formulario, la ruta y la cadena de consulta, construye el objeto del parámetro y ejecuta las validaciones.
7. La acción devuelve un `ActionResult`; si es un `ViewResult`, el **Motor Razor** compila la plantilla `.cshtml` a HTML y lo escribe en la respuesta.

Kestrel vs. IIS: En .NET 8, Kestrel es el servidor web embebido por defecto. IIS actúa como *reverse proxy* opcional. Kestrel implementa los protocolos HTTP/1.1, HTTP/2 y HTTP/3 directamente, sin depender del servidor del sistema operativo, lo que lo hace idóneo para contenedores Docker y entornos Linux [1].

2.2. Data Annotations: mecanismo interno y jerarquía de atributos

Los **Data Annotations** pertenecen al espacio de nombres `System.ComponentModel.DataAnnotations` y constituyen un conjunto de atributos C# que decoran las propiedades de un modelo para expresar restricciones de datos de forma declarativa [4].

La clase abstracta `ValidationAttribute` es la raíz jerárquica. Define el método virtual `IsValid(object? value, ValidationContext ctx)`, que retorna un `ValidationResult` (exitoso o con mensaje de error). En ASP.NET Core, la validación se desencadena automáticamente durante el *Model Binding*: la clase interna `DefaultObjectValidator` llama a `Validator.TryValidateObject(instance, ctx, results, true)`, que mediante reflexión descubre todos los `ValidationAttribute` en cada propiedad, los invoca y acumula los resultados en el diccionario `ModelState` [5].

La Tabla 1 describe los atributos empleados en esta práctica:

Cuadro 1: Atributos de Data Annotations utilizados en `Producto.cs`.

Atributo	Comportamiento interno
[Required]	Sobreescribe <code>IsValid()</code> para rechazar <code>null</code> , <code>string.Empty</code> y cadenas de solo espacios en blanco (<code>AllowEmptyStrings</code> desactivado por defecto).
[StringLength]	Evalúa <code>string.Length</code> contra <code>MaximumLength</code> y <code>MinimumLength</code> ; lanza <code>ValidationResult</code> si la longitud está fuera del rango declarado.
[Range]	Convierte el valor al tipo subyacente y lo compara mediante <code>IComparable</code> ; soporta enteros, decimales, fechas y <code>double</code> .
[RegularExpression]	Aplica <code>Regex.IsMatch()</code> con el patrón indicado; internamente utiliza <code>RegexOptions.None</code> por defecto.
[DataType]	Anotación semántica que no valida por sí sola; informa al motor de vistas cómo formatear el campo (e.g., <code>DataType.Currency</code> activa el prefijo del símbolo monetario según la cultura activa).
[Display]	Proporciona metadatos de presentación (<code>Name</code> , <code>Prompt</code> , <code>Description</code>) que los <i>tag helpers</i> <code>asp-for</code> leen mediante reflexión para generar etiquetas HTML legibles.

Cuando alguna validación falla, la entrada se registra en `ModelState["NombrePropiedad"].Errors`, una colección de objetos `ModelError` que contienen el mensaje de error y la excepción de conversión de tipo si aplica. La propiedad `ModelState.IsValid` retorna `true` únicamente si **todas** las entradas son válidas y **todos** los valores se convirtieron exitosamente al tipo esperado.

2.3. Model Binding: del protocolo HTTP al objeto C#

El **Model Binding** es el proceso mediante el cual ASP.NET Core deserializa los datos de una petición HTTP hacia los parámetros tipados de una acción. La clase `ComplexObjectModelBinder` procesa tipos complejos (clases con múltiples propiedades) del siguiente modo [5]:

1. **Selección del *ValueProvider*:** se consultan los proveedores de valores en orden de prioridad descendente: `FormValueProvider` (cuerpo `application/x-www-form-urlencoded`), `RouteValueProvider` (segmentos de la URL) y `QueryStringValueProvider` (parámetros GET). Cada uno implementa `IValueProvider` con los métodos `ContainsPrefix()` y `GetValue()`.
2. **Iteración de propiedades:** mediante reflexión, se itera sobre cada propiedad pública y con setter del tipo objetivo, buscando una clave coincidente en los proveedores (e.g., `Nombre`, `Precio`).
3. **Conversión de tipos:** cada valor (siempre una cadena en HTTP) se convierte al tipo C# de la propiedad usando `TypeConverter` o `IModelBinder` especializados (e.g., `DecimalModelBinder` para decimal).
4. **Asignación y validación:** el valor convertido se asigna a la propiedad y se ejecutan los `ValidationAttribute`. Los errores se acumulan en `ModelState`.

Seguridad en el binding: El atributo `[Bind]` permite especificar una lista blanca de propiedades aceptadas (`[Bind("Nombre,Precio,Stock")]`) para prevenir *overposting* o *mass assignment*, un vector de ataque donde el cliente envía campos no esperados (e.g., `Id` o campos de rol de usuario) que el binder asignaría automáticamente si no se limita explícitamente [2].

2.4. Validación en servidor frente a validación en cliente

2.4.1. Validación en el servidor (*server-side*)

La validación en servidor se ejecuta dentro del pipeline de ASP.NET Core, en la fase de *Model Binding*, antes de que el cuerpo de la acción se ejecute. Es **imposible de eludir** desde el cliente, ya que opera directamente sobre el objeto HTTP procesado por Kestrel. El patrón de uso es:

Listing 1: Patrón estándar de validación server-side.

```
1 [HttpPost]
2 [ValidateAntiForgeryToken]
3 public IActionResult Create(Producto producto)
4 {
5     if (ModelState.IsValid)
6     {
7         // Los datos son validos: procesar negocio
8         return RedirectToAction(nameof(Index));
9     }
10    // ModelState contiene errores: reenviar la vista
11    return View(producto);
12 }
```

2.4.2. Validación en el cliente (*client-side*)

ASP.NET Core MVC integra *jQuery Validation* con el plugin *Unobtrusive Validation*. Cuando los *tag helpers* `asp-for` y `asp-validation-for` renderizan los campos del formulario, generan atributos HTML5 `data-val-*` que el plugin JavaScript lee para ejecutar la validación en el navegador *sin* una petición al servidor [4]:

Listing 2: HTML generado por `asp-for` / `asp-validation-for`.

```
1 <!-- HTML generado automaticamente por el tag helper -->
2 <input
3     class="form-control"
4     type="text"
5     id="Nombre" name="Nombre"
6     data-val="true"
7     data-val-required="El nombre del producto es obligatorio."
8     data-val-length="El nombre debe tener entre 2 y 100 caracteres."
9     data-val-length-max="100"
10    data-val-length-min="2">
11 <span
12     class="text-danger small field-validation-valid"
13     data-valmsg-for="Nombre"
14     data-valmsg-replace="true">
15 </span>
```

Regla de oro de la seguridad: La validación en cliente es una optimización de usabilidad, no un mecanismo de seguridad. Un atacante puede deshabilitar JavaScript, modificar los atributos `data-val-*` con las herramientas de desarrollo del navegador o enviar una petición HTTP cruda con `curl` o Postman, evadiendo cualquier validación del lado del cliente. La validación en servidor con `ModelState.IsValid` es, por tanto, **no negociable** en cualquier aplicación web [6].

2.5. Protección CSRF y el mecanismo *AntiForgeryToken*

2.5.1. Naturaleza del ataque CSRF

Un ataque **Cross-Site Request Forgery** (CSRF, también denominado XSRF) ocurre cuando un sitio malicioso induce al navegador de una víctima autenticada a emitir peticiones no deseadas hacia una aplicación web legítima. El ataque aprovecha que el navegador incluye automáticamente las *cookies* de sesión en toda petición al origen correspondiente, independientemente del origen HTML que lanzó dicha petición [7].

Ejemplo de ataque: si el usuario está autenticado en `banca.com`, un sitio malicioso puede embeber `` y el navegador ejecutará la petición con la cookie de sesión válida.

2.5.2. El patrón *Synchronizer Token* en ASP.NET Core

ASP.NET Core implementa el **patrón Synchronizer Token** a través del servicio `IAntiforgery` [8]. El mecanismo opera en dos fases:

Fase de generación (cuando se renderiza el formulario GET):

1. `FormTagHelper` detecta el atributo `asp-antiforgery="true"` y llama a `IAntiforgery.GetAndSynchronizerToken()`.
2. El servicio genera dos tokens criptográficamente vinculados: un **token de cookie** (aleatorio) y un **token de campo** (deriva del token de cookie más un nonce de un solo uso), ambos cifrados con la *Data Protection Stack* de ASP.NET Core (AES-256-CBC + HMACSHA256 mediante `IDataProtector`).
3. El token de cookie se almacena en la cookie HTTP `.AspNetCore.Antiforgery.[hash]`; el token de campo se inserta como `<input type="hidden" name="__RequestVerificationToken" value="token">`.

Fase de validación (cuando llega el POST):

1. El atributo `[ValidateAntiForgeryToken]` registra un `ValidateAntiForgeryTokenAuthorizationFilter`.
2. El filtro llama a `IAntiforgery.ValidateRequestAsync()`, que extrae y descifra ambos tokens.
3. Verifica que el componente aleatorio del token de campo coincida criptográficamente con el token de cookie, y que el nonce de campo no haya sido utilizado previamente.
4. Si la validación falla, se devuelve HTTP 400 Bad Request; si pasa, la ejecución continúa.

Tag helper vs. HTML helper: Las expresiones `asp-antiforgery="true"` (tag helper declarativo) y `@Html.AntiForgeryToken()` (HTML helper imperativo) pro-

ducen **exactamente el mismo** `<input>` oculto. La convención moderna en ASP.NET Core 6+ prefiere los *tag helpers* por ser más legibles en el contexto HTML y estar integrados con IntelliSense en los editores [9].

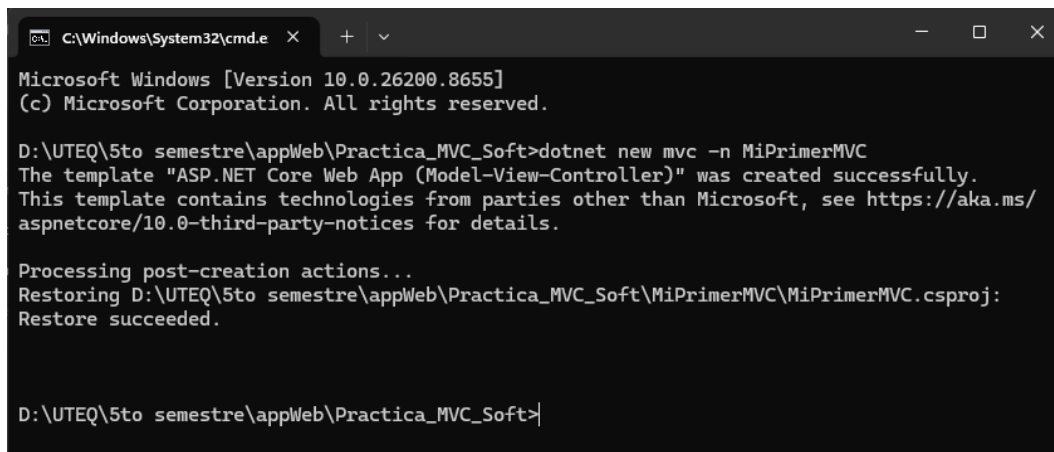
3. Desarrollo e Implementación

3.1. Fase 1: Inicialización del proyecto con .NET CLI

La *.NET Command-Line Interface* (CLI) proporciona la herramienta `dotnet` para gestionar el ciclo de vida completo de una aplicación .NET: creación, compilación, prueba y publicación. La plantilla `mvc` genera un proyecto preconfigurado con Bootstrap 5, el pipeline de middlewares estándar y la estructura de carpetas del patrón MVC.

Listing 3: Creación y ejecución inicial del proyecto MVC.

```
1 # Verificar version del SDK instalado
2 dotnet --version
3 # Resultado esperado: 8.x.x
4
5 # Crear el proyecto con la plantilla MVC
6 dotnet new mvc -n MiPrimerMVC
7
8 # Ingresar al directorio generado
9 cd MiPrimerMVC
10
11 # Restaurar paquetes NuGet e iniciar la aplicacion
12 dotnet run
13 # Resultado: Now listening on: https://localhost:XXXX
```



The screenshot shows a Windows Command Prompt window with the following text:

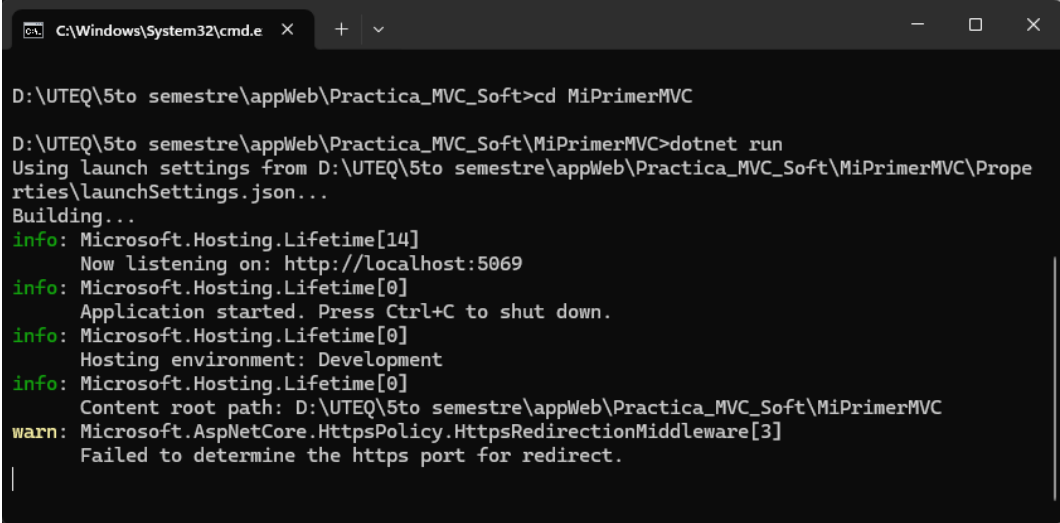
```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

D:\UTEQ\5to semestre\appWeb\Practica_MVC_Soft>dotnet new mvc -n MiPrimerMVC
The template "ASP.NET Core Web App (Model-View-Controller)" was created successfully.
This template contains technologies from parties other than Microsoft, see https://aka.ms/aspnetcore/10.0-third-party-notices for details.

Processing post-creation actions...
Restoring D:\UTEQ\5to semestre\appWeb\Practica_MVC_Soft\MiPrimerMVC\MiPrimerMVC.csproj:
Restore succeeded.

D:\UTEQ\5to semestre\appWeb\Practica_MVC_Soft>
```

Figura 1: Terminal mostrando la salida completa de `dotnet new mvc -n MiPrimerMVC`, con la lista de archivos generados y la confirmación *Restore succeeded*.



```
C:\Windows\System32\cmd.e X + v

D:\UTEQ\5to semestre\appWeb\Practica_MVC_Soft>cd MiPrimerMVC

D:\UTEQ\5to semestre\appWeb\Practica_MVC_Soft\MiPrimerMVC>dotnet run
Using launch settings from D:\UTEQ\5to semestre\appWeb\Practica_MVC_Soft\MiPrimerMVC\Properties\launchSettings.json...
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5069
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\UTEQ\5to semestre\appWeb\Practica_MVC_Soft\MiPrimerMVC
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]
      Failed to determine the https port for redirect.
```

Figura 2: Terminal con la salida de `dotnet run`, indicando el puerto en escucha (Now listening on: `https://localhost:XXXX`) y el mensaje `Application started. Press Ctrl+C to shut down`.

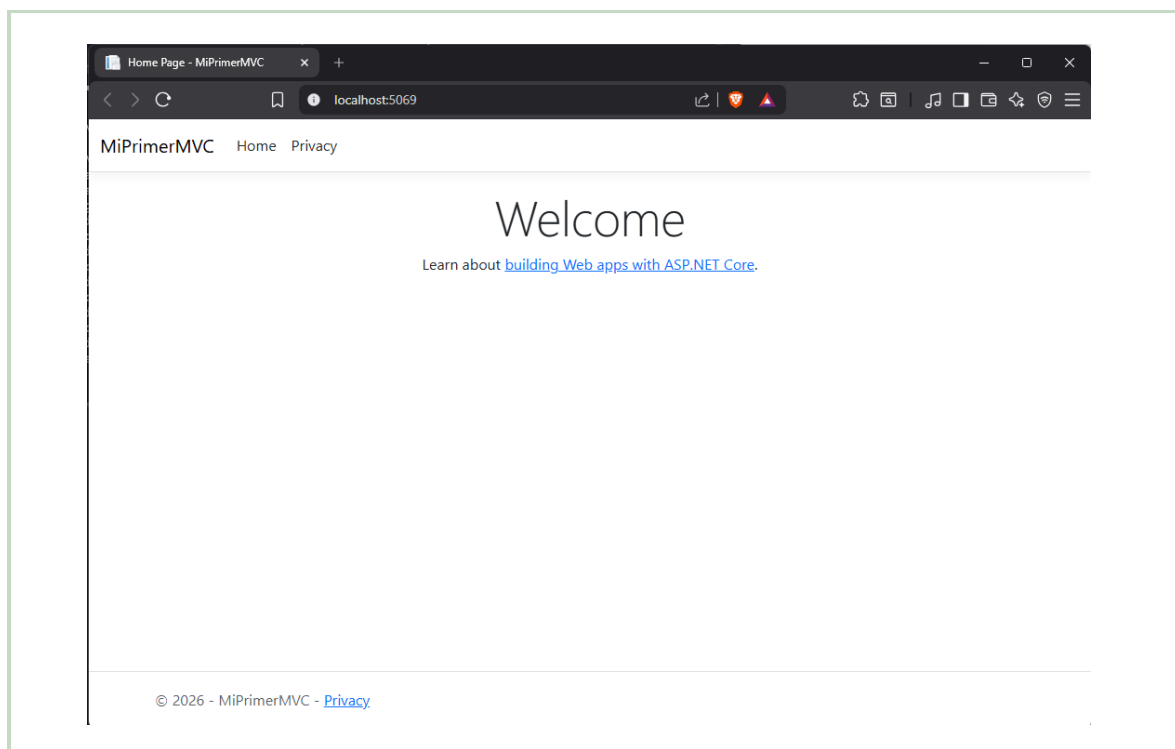


Figura 3: Navegador en `https://localhost:XXXX` mostrando la página de bienvenida por defecto de ASP.NET Core MVC, con el encabezado `Welcome` y los enlaces `Home` y `Privacy` en el navbar.

La estructura de carpetas generada por la plantilla es la siguiente:

Listing 4: Estructura de directorios del proyecto `MiPrimerMVC`.

```

1 MiPrimerMVC/
2   Controllers/          <- Controladores (HomeController.cs)
3   Models/              <- Modelos de datos
4   Views/               <- Vistas Razor (.cshtml)
5     Home/
6     Shared/
7   wwwroot/             <- Archivos estaticos (CSS, JS, imagenes)
8   Program.cs           <- Punto de entrada y pipeline
9   appsettings.json     <- Configuracion de la aplicacion
10  MiPrimerMVC.csproj   <- Metadatos y dependencias NuGet

```

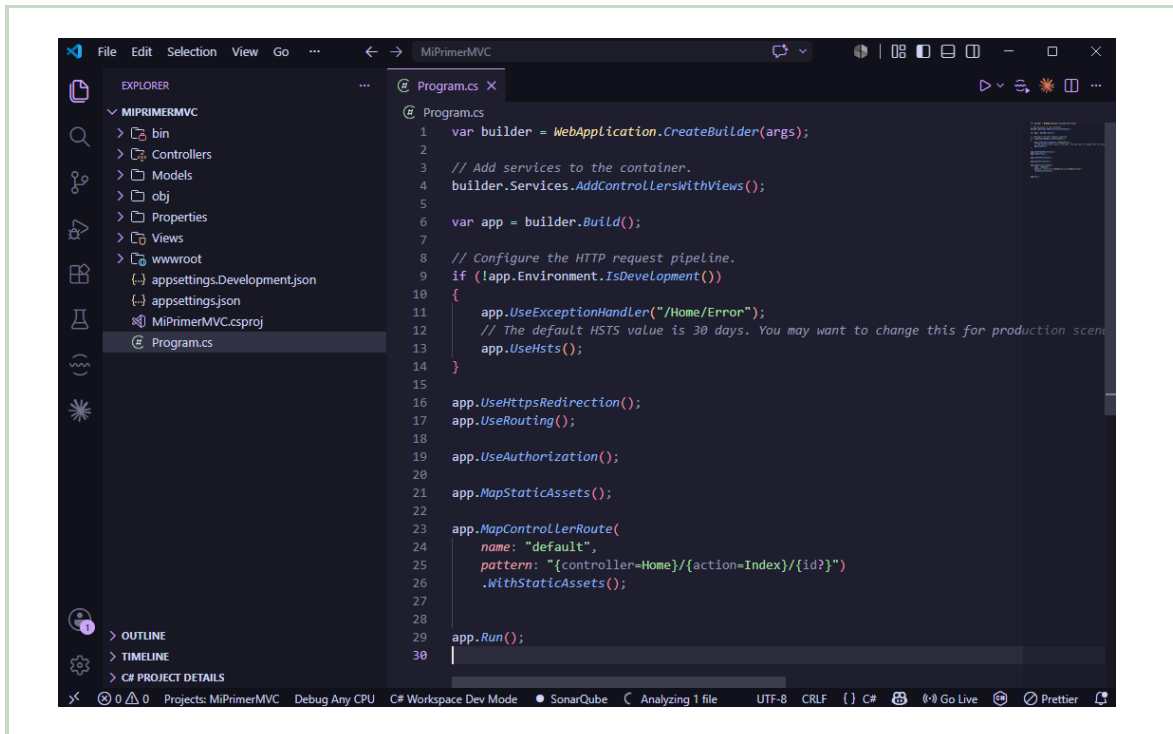


Figura 4: Explorador de archivos en VS Code mostrando la estructura completa generada, incluyendo Controllers/, Models/, Views/Shared/_Layout.cshtml y Program.cs.

3.2. Fase 2: Modelo Producto con Data Annotations

El modelo concentra toda la lógica de restricciones de datos mediante atributos declarativos. Se crea el archivo Models/Producto.cs:

Listing 5: Models/Producto.cs — Modelo con validaciones declarativas.

```

1 using System.ComponentModel.DataAnnotations;
2
3 namespace MiPrimerMVC.Models
4 {
5     /// <summary>
6     /// Modelo de dominio que representa un producto del catalogo.
7     /// Las Data Annotations declaran las restricciones de validacion
8     /// que el pipeline de ASP.NET Core evaluara automaticamente
9     /// durante el Model Binding antes de ejecutar la accion POST.
10    /// </summary>
11    public class Producto

```

```
12 {
13     // Identificador generado por el repositorio; no se valida.
14     public int Id { get; set; }
15
16     [Required(ErrorMessage = "El nombre del producto es obligatorio.")]
17     [StringLength(100, MinimumLength = 2,
18         ErrorMessage = "El nombre debe tener entre 2 y 100 caracteres.")]
19     [Display(Name = "Nombre del Producto")]
20     public string Nombre { get; set; } = string.Empty;
21
22     [Required(ErrorMessage = "La descripcion es obligatoria.")]
23     [StringLength(500,
24         ErrorMessage = "La descripcion no puede superar los 500 caracteres.")]
25     [Display(Name = "Descripcion")]
26     public string Descripcion { get; set; } = string.Empty;
27
28     [Required(ErrorMessage = "El precio es obligatorio.")]
29     [Range(0.01, 99999.99,
30         ErrorMessage = "El precio debe estar entre $0.01 y $99,999.99.")]
31     [DataType(DataType.Currency)]
32     [Display(Name = "Precio (USD)")]
33     public decimal Precio { get; set; }
34
35     [Required(ErrorMessage = "El stock es obligatorio.")]
36     [Range(0, 10000,
37         ErrorMessage = "El stock debe estar entre 0 y 10,000 unidades.")]
38     [Display(Name = "Stock disponible")]
39     public int Stock { get; set; }
40
41     [Required(ErrorMessage = "La categoria es obligatoria.")]
42     [StringLength(50,
43         ErrorMessage = "La categoria no puede superar los 50 caracteres.")]
44     [Display(Name = "Categoria")]
45     public string Categoria { get; set; } = string.Empty;
46 }
47 }
```

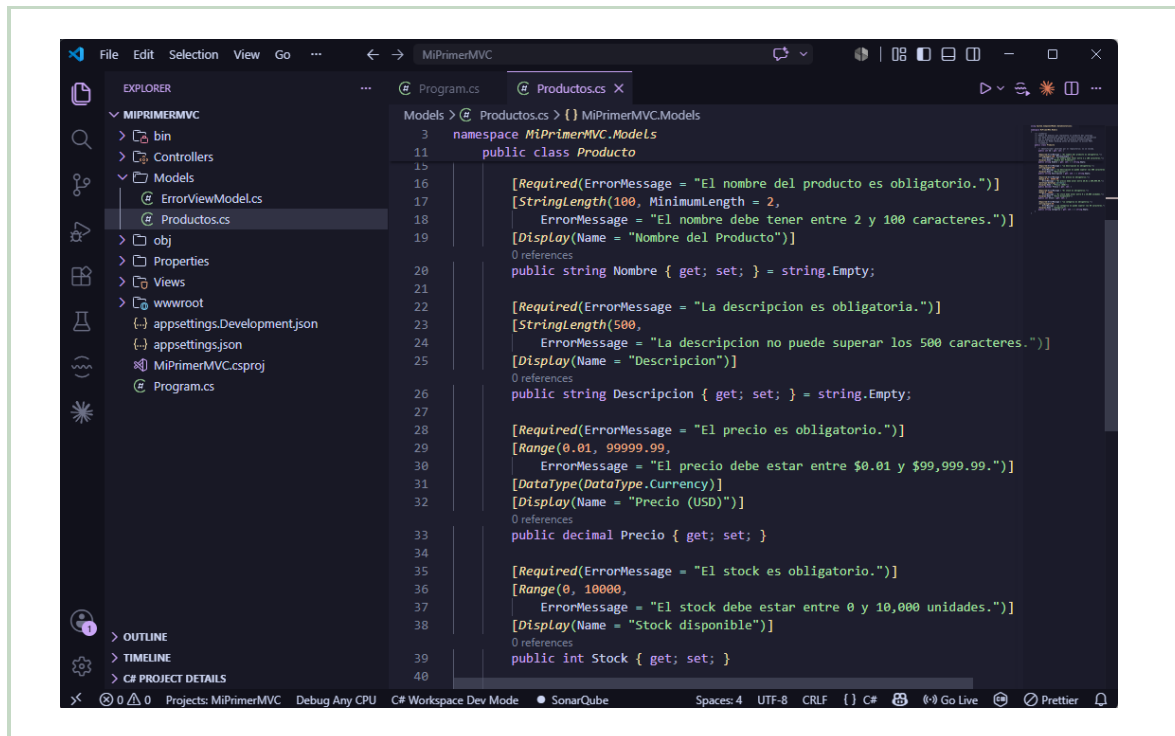


Figura 5: Editor de código con *Models/Producto.cs* completamente visible, mostrando los atributos *[Required]*, *[StringLength]*, *[Range]*, *[DataType]* y *[Display]* decorando cada propiedad del modelo.

3.3. Fase 3: ProductoController con acciones GET y POST

El controlador encapsula la lógica de flujo de la aplicación. En este escenario de demostración, se usa un repositorio estático en memoria (`static List<Producto>`) que simula la capa de persistencia sin requerir una base de datos.

Listing 6: *Controllers/ProductoController.cs* — Controlador con acción GET e POST.

```

1 using Microsoft.AspNetCore.Mvc;
2 using MiPrimerMVC.Models;
3
4 namespace MiPrimerMVC.Controllers
5 {
6     /// <summary>
7     /// Controlador responsable de las operaciones CRUD sobre Producto.
8     /// Hereda de Controller (no ControllerBase) para tener acceso
9     /// a la API de Vistas (View(), TempData, ModelState, etc.).
10    /// </summary>
11    public class ProductoController : Controller
12    {
13        // -----
14        // Repositorio en memoria (solo para fines didacticos).
15        // En produccion se inyectaria IProductoRepository via DI.
16        // -----
17        private static readonly List<Producto> _productos = new();
18        private static int _nextId = 1;
19
20        // -----

```

```
21 // GET: /Producto/Index
22 // Recupera todos los productos y los pasa como modelo a la vista.
23 // -----
24 public IActionResult Index()
25 {
26     return View(_productos);
27 }
28
29 // -----
30 // GET: /Producto/Create
31 // Devuelve el formulario vacio para crear un nuevo producto.
32 // -----
33 [HttpGet]
34 public IActionResult Create()
35 {
36     return View();
37 }
38
39 // -----
40 // POST: /Producto/Create
41 //
42 // [ValidateAntiForgeryToken] registra un filtro de autorizacion
43 // que llama a IAntiforgery.ValidateRequestAsync() ANTES de que
44 // este metodo se ejecute. Si el token no es valido, devuelve
45 // HTTP 400 Bad Request inmediatamente.
46 //
47 // El parametro 'producto' es poblado por el Model Binder:
48 // ComplexObjectModelBinder itera las propiedades del tipo,
49 // extrae los valores del FormValueProvider y ejecuta las
50 // validaciones. ModelState refleja el resultado.
51 // -----
52 [HttpPost]
53 [ValidateAntiForgeryToken]
54 public IActionResult Create(Producto producto)
55 {
56     if (ModelState.IsValid)
57     {
58         // Todos los ValidationAttribute pasaron: persistir.
59         producto.Id = _nextId++;
60         _productos.Add(producto);
61
62         // TempData persiste un mensaje entre el redirect (PRG pattern).
63         TempData["Exito"] =
64             $"Producto '{producto.Nombre}' registrado correctamente.";
65
66         // Post/Redirect/Get pattern: evita el reenvio del formulario
67         // al recargar la pagina.
68         return RedirectToAction(nameof(Index));
69     }
70
71     // Al menos un ValidationAttribute fallo.
72     // Se reenvía la vista con el objeto 'producto' (datos ingresados)
73     // para que los tag helpers rendericen los mensajes de error
74     // usando ModelState["Campo"].Errors.
75     return View(producto);
76 }
77 }
78 }
```

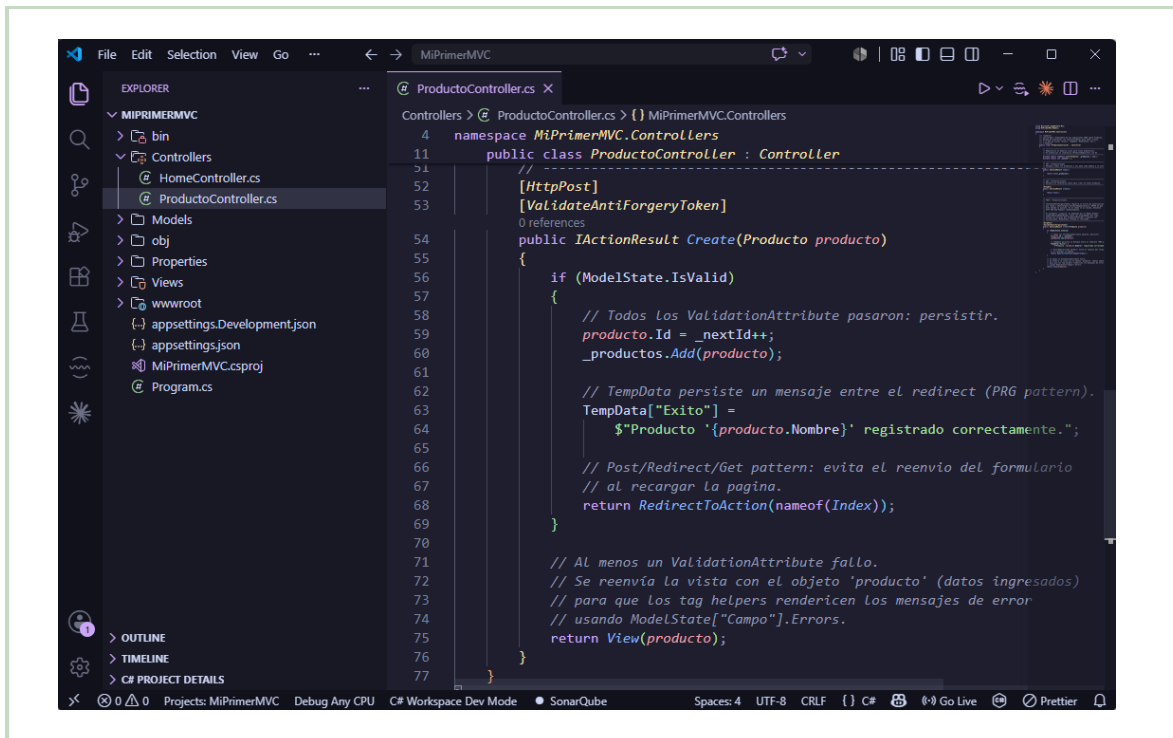


Figura 6: Editor con `Controllers/ProductoController.cs` mostrando ambas acciones `Create`: la `GET` simple y la `POST` con los atributos `[HttpPost]` y `[ValidateAntiForgeryToken]`, y el bloque `if (ModelState.IsValid)` claramente visibles.

3.4. Fase 4: Vista Razor `Index.cshtml`

La vista de listado recibe `IEnumerable<Producto>` como modelo fuertemente tipado y renderiza una tabla Bootstrap. El mensaje de éxito se lee de `TempData`, que persiste durante un solo ciclo de petición (patrón PRG).

Listing 7: `Views/Producto/Index.cshtml` — Vista de listado.

```

1 @model IEnumerable<MiPrimerMVC.Models.Producto>
2 @{
3     ViewData["Title"] = "Catalogo de Productos";
4 }
5
6 <div class="container mt-4">
7     <h1 class="mb-3">Catalogo de Productos</h1>
8
9     * Banner de exito mostrado solo tras un redirect exitoso (TempData) *
10    @if (TempData["Exito"] != null)
11    {
12        <div class="alert alert-success alert-dismissible fade show"
13            role="alert">
14            <strong>#10004;</strong> @TempData["Exito"]
15            <button type="button" class="btn-close"
16                data-bs-dismiss="alert" aria-label="Close"></button>
17        </div>
18    }
19
20    <a asp-action="Create" class="btn btn-primary mb-3">

```

```
21     + Agregar Producto
22   </a>
23
24   @if (!Model.Any())
25   {
26     <div class="alert alert-info">
27       No hay productos registrados aun. Haga clic en
28       <strong>Agregar Producto</strong> para comenzar.
29     </div>
30   }
31   else
32   {
33     <table class="table table-bordered table-hover align-middle">
34       <thead class="table-dark">
35         <tr>
36           <th>ID</th>
37           <th>Nombre</th>
38           <th>Descripcion</th>
39           <th>Precio</th>
40           <th>Stock</th>
41           <th>Categoria</th>
42         </tr>
43       </thead>
44       <tbody>
45         @foreach (var p in Model)
46         {
47           <tr>
48             <td>@p.Id</td>
49             <td>@p.Nombre</td>
50             <td>@p.Descripcion</td>
51             <td>@p.Precio.ToString("C")</td>
52             <td>@p.Stock</td>
53             <td>@p.Categoria</td>
54           </tr>
55         }
56       </tbody>
57     </table>
58   }
59 </div>
```

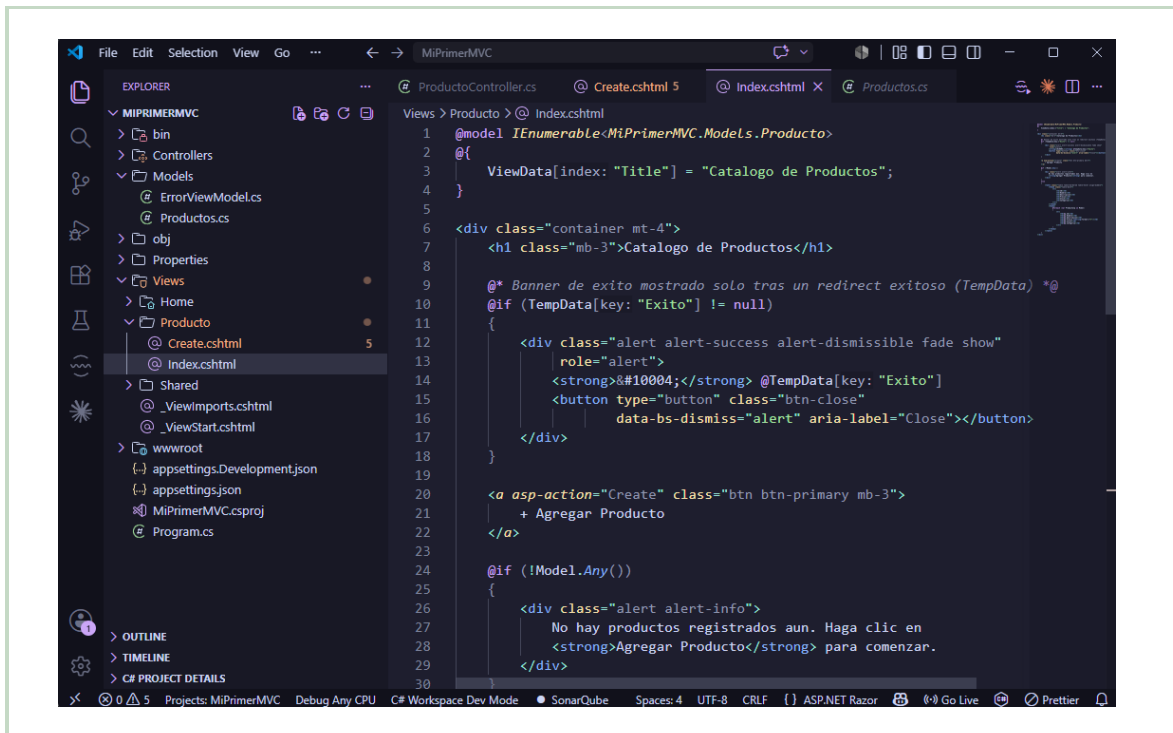


Figura 7: Editor con Views/Producto/Index.cshtml mostrando la directiva `@model IEnumerable<...>`, el bloque condicional `TempData["Exito"]` y el bucle `@foreach` que construye las filas de la tabla.

3.5. Fase 5: Vista Razor Create.cshtml con AntiForgeryToken

Esta es la vista más importante de la práctica. El atributo `asp-antiforgery="true"` en la etiqueta `<form>` instruye al `FormTagHelper` para que llame a `IAntiforgery.GetAndStoreTokens()` e inserte el campo oculto con el token. Los `tag helpers` `asp-validation-for` generan los atributos `data-val-*` necesarios para la validación en cliente mediante `jQuery Validation` [9].

Listing 8: Views/Producto/Create.cshtml — Formulario con protección CSRF y validación reactiva.

```

1 @model MiPrimerMVC.Models.Producto
2 @{
3     ViewData["Title"] = "Registrar Nuevo Producto";
4 }
5
6 <div class="container mt-4" style="max-width: 700px;">
7     <h1>Registrar Nuevo Producto</h1>
8     <hr />
9
10    @*
11        asp-action      : el FormTagHelper genera action="/Producto/Create"
12        asp-antiforgery="true" : inyecta el campo oculto __RequestVerificationToken
13                               y la cookie correspondiente.
14        method="post" : asegura que los datos viajen en el cuerpo HTTP.
15    *@
16    <form asp-action="Create" asp-antiforgery="true" method="post">
17
18        <div class="mb-3">
```



```

19     @* asp-for lee [Display(Name="...")] del modelo para generar
20     el label y los atributos name/id del input. *@
21     <label asp-for="Nombre" class="form-label fw-bold"></label>
22     <input asp-for="Nombre" class="form-control"
23         placeholder="Ej: Laptop Lenovo ThinkPad" />
24     @* asp-validation-for genera el span con data-valmsg-for
25     que jQuery Validation popula con los mensajes de error. *@
26     <span asp-validation-for="Nombre"
27         class="text-danger small"></span>
28 </div>
29
30 <div class="mb-3">
31     <label asp-for="Descripcion" class="form-label fw-bold"></label>
32     <textarea asp-for="Descripcion" class="form-control"
33         rows="3"
34         placeholder="Descripcion del producto...">
35     </textarea>
36     <span asp-validation-for="Descripcion"
37         class="text-danger small"></span>
38 </div>
39
40 <div class="row">
41     <div class="col-md-6 mb-3">
42         <label asp-for="Precio" class="form-label fw-bold"></label>
43         <input asp-for="Precio" class="form-control"
44             type="number" step="0.01" min="0.01"
45             placeholder="0.00" />
46         <span asp-validation-for="Precio"
47             class="text-danger small"></span>
48     </div>
49     <div class="col-md-6 mb-3">
50         <label asp-for="Stock" class="form-label fw-bold"></label>
51         <input asp-for="Stock" class="form-control"
52             type="number" min="0"
53             placeholder="0" />
54         <span asp-validation-for="Stock"
55             class="text-danger small"></span>
56     </div>
57 </div>
58
59 <div class="mb-3">
60     <label asp-for="Categoria" class="form-label fw-bold"></label>
61     <input asp-for="Categoria" class="form-control"
62         placeholder="Ej: Electronica" />
63     <span asp-validation-for="Categoria"
64         class="text-danger small"></span>
65 </div>
66
67 <div class="d-flex gap-2 mt-3">
68     <button type="submit" class="btn btn-success px-4">
69         Guardar Producto
70     </button>
71     <a asp-action="Index" class="btn btn-secondary">
72         Cancelar
73     </a>
74 </div>
75
76 </form>

```

```

77 </div>
78
79 /* La seccion Scripts carga jQuery Validation + Unobtrusive Validation,
80    necesarios para la validacion en cliente (client-side). */
81 @section Scripts {
82     @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
83 }

```

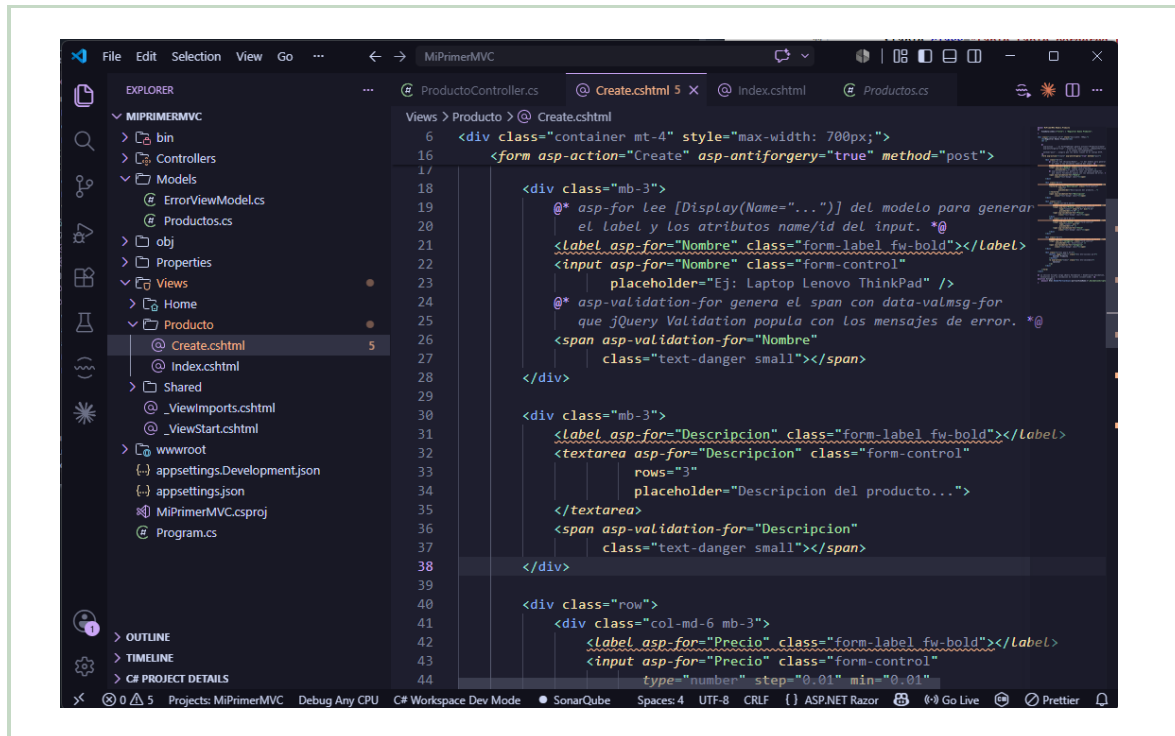


Figura 8: Editor con Views/Producto/Create.cshtml mostrando la línea `<form asp-action=Create. asp-antiforgery="true">` y los tag helpers `asp-validation-for` debajo de cada campo del formulario.

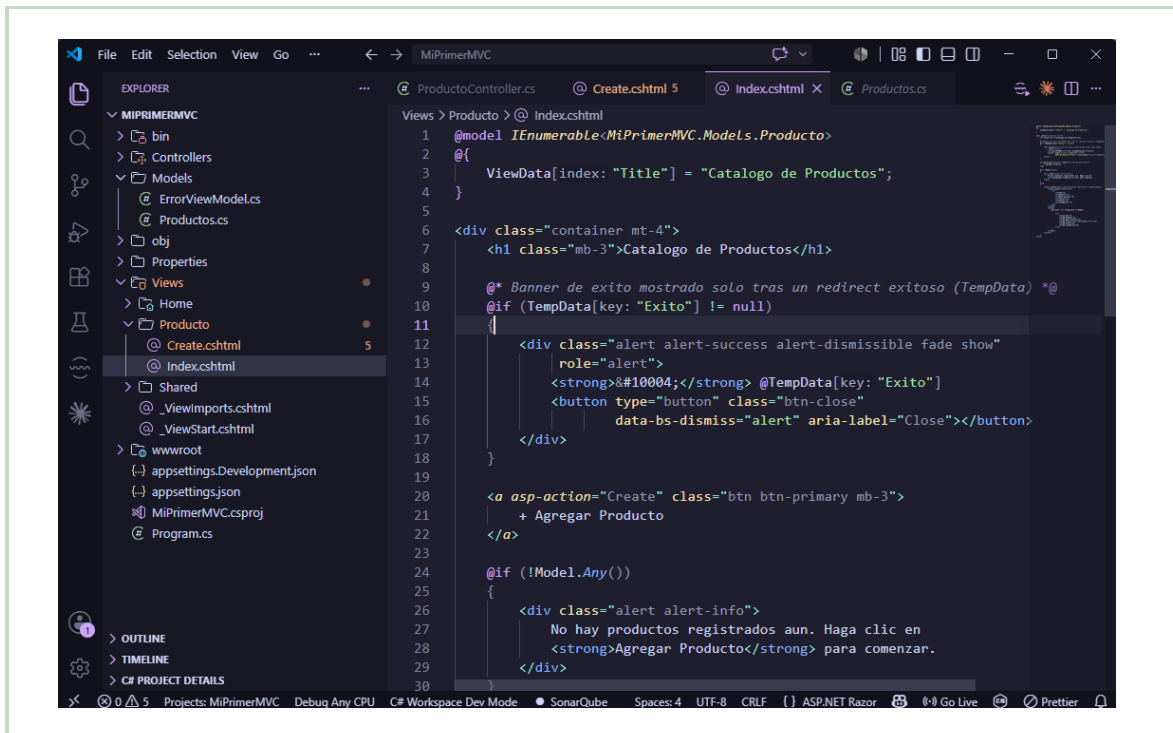


Figura 9: Explorador de archivos de VS Code con el proyecto completo: *Models/Producto.cs*, *Controllers/ProductoController.cs*, *Views/Producto/Create.cshtml* e *Views/Producto/Index.cshtml* visibles en el árbol de directorios.

3.6. Fase 6: Configuración del pipeline en Program.cs

El archivo *Program.cs* generado por la plantilla ya incluye todos los middlewares necesarios. Se presenta para que el lector comprenda el pipeline completo:

Listing 9: *Program.cs* — Pipeline de middlewares de la aplicación.

```

1 var builder = WebApplication.CreateBuilder(args);
2
3 // ----- Registro de servicios (Composition Root) -----
4 // AddControllersWithViews registra en el DI container:
5 // - IControllerFactory, IViewEngine (Razor), IModelMetadataProvider,
6 // - IAntiforgery, IModelBinderFactory, IObjectModelValidator, etc.
7 builder.Services.AddControllersWithViews();
8
9 var app = builder.Build();
10
11 // ----- Pipeline de middlewares (orden obligatorio) -----
12 if (!app.Environment.IsDevelopment())
13 {
14     app.UseExceptionHandler("/Home/Error");
15     app.UseHsts(); // Fuerza HTTPS en produccion
16 }
17
18 app.UseHttpsRedirection(); // Redirige HTTP -> HTTPS
19 app.UseStaticFiles(); // Sirve wwwroot/ (CSS, JS, imagenes)
20 app.UseRouting(); // Construye la tabla de rutas
21 app.UseAuthorization(); // Middleware de autorizacion
22

```

```
23 // Ruta convencional: /{controller}/{action}/{id?}
24 app.MapControllerRoute(
25     name: "default",
26     pattern: "{controller=Home}/{action=Index}/{id?}");
27
28 app.Run();
```

4. Verificación con Casos de Prueba

Se ejecutan cinco casos de prueba que cubren los escenarios de validación más representativos. Se navega a `https://localhost:XXXX/Producto/Create` tras ejecutar `dotnet run`.

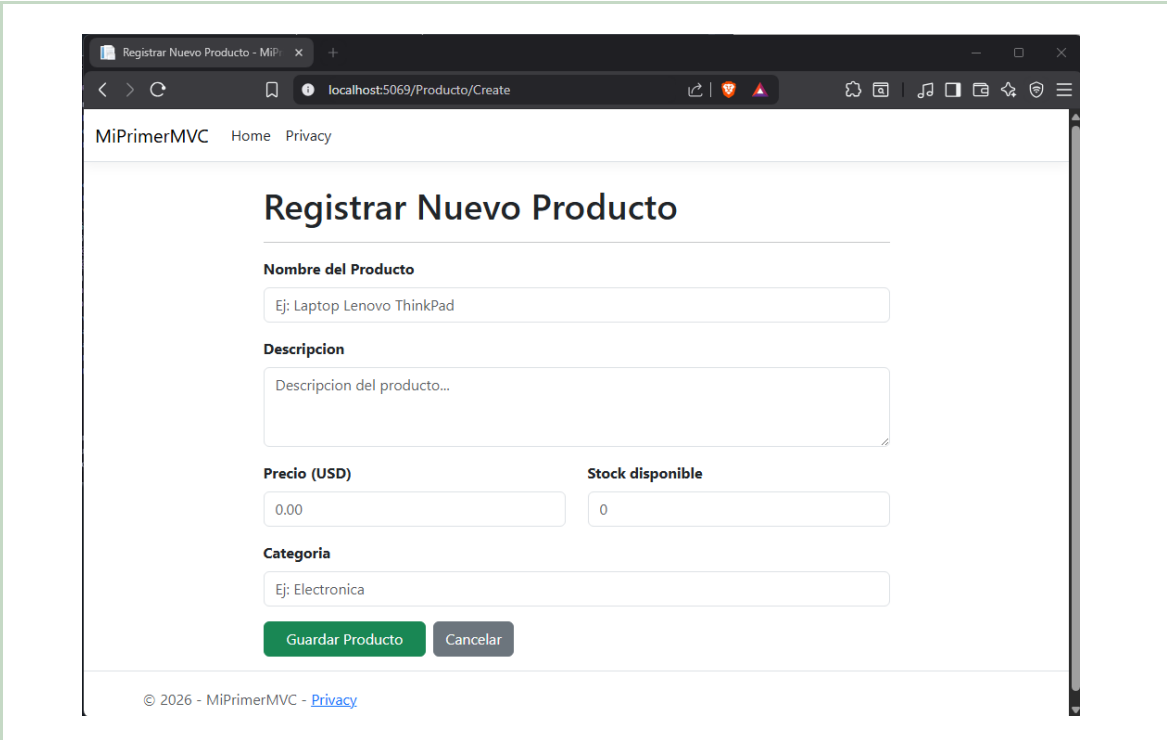


Figura 10: Navegador mostrando el formulario `/Producto/Create` con todos los campos vacíos y los botones *Guardar Producto* y *Cancelar* visibles. No hay errores porque el formulario aún no se ha enviado.

4.1. Caso 1 — Todos los campos vacíos

Datos de entrada: Se hace clic en *Guardar Producto* sin rellenar ningún campo.

Resultado esperado: `ModelState.IsValid = false`; cinco mensajes de error, uno por cada propiedad anotada con `[Required]`.

ASP.NET Core MVC Resultado servidor:

Figura 11: Formulario después del POST vacío mostrando los cinco mensajes de error en rojo: “El nombre del producto es obligatorio.”, “La descripción es obligatoria.”, “El precio es obligatorio.”, etc. El tag helper `asp-validation-for` inyectó los spans con los textos extraídos de `ModelState`.

Mecanismo interno: Al recibir el POST con todos los campos vacíos, el `ComplexObjectModelBinder` intenta convertir las cadenas vacías al tipo de cada propiedad. Para `decimal Precio` y `int Stock`, la conversión falla y se agrega un error de *tipo* en `ModelState` además del error de `[Required]`. Razor renderiza todos los errores de la colección `ModelState[Campo].Errors` mediante el tag helper `asp-validation-summary`.

4.2. Caso 2 — Valores numéricos fuera de rango

Datos de entrada: Nombre=*Producto Test*, Descripción=*Descripción de prueba*, Precio=*-5.00*, Stock=*99999*, Categoría=*Prueba*.

Resultado esperado: Dos errores del atributo `[Range]`: uno en Precio (inferior a 0.01) y uno en Stock (superior a 10000).

ASP.NET Core MVC Resultado servidor:

Registrar Nuevo Producto

Nombre del Producto

Laptop Lenovo Thinkpad

Descripcion

Laptop de oficina con procesador ultra i9 de última generación

Precio (USD)

-5

Please enter a value greater than or equal to 0.01.

Stock disponible

99999

El stock debe estar entre 0 y 10,000 unidades.

Categoria

Electronica

Guardar Producto Cancelar

© 2026 - MiPrimerMVC - [Privacy](#)

Figura 12: Formulario tras el POST con valores numéricos fuera de rango. El campo Precio muestra “El precio debe estar entre \$0.01 y \$99,999.99” y el campo Stock muestra “El stock debe estar entre 0 y 10,000 unidades”. Los demás campos se conservan con los valores enviados.

4.3. Caso 3 — Inspección del token CSRF en DevTools

Procedimiento: Se abre Chrome DevTools (F12) en la pestaña *Network* antes de enviar el formulario con datos válidos. Se hace clic en *Guardar Producto* y se selecciona la petición POST en el panel de red.

Resultado esperado: El payload de la petición POST contiene el campo oculto `__RequestVerificationToken` con un valor criptográfico extenso (generado por `IAntiforgery`), además de los campos del formulario.

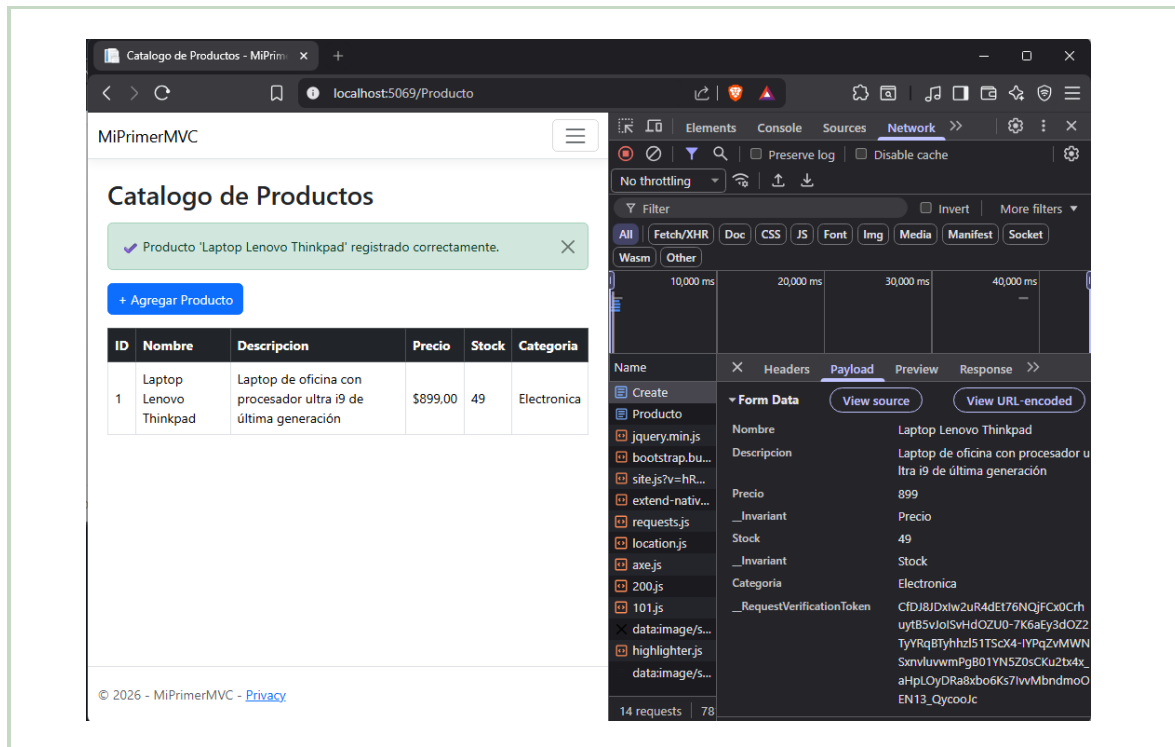


Figura 13: Chrome DevTools → Network → petición POST a /Producto/Create → pestaña Payload, mostrando todos los campos del formulario y el campo `__RequestVerificationToken` con su valor cifrado largo, demostrando que `asp-antiforgery="true"` inyectó el token correctamente.

Prueba de seguridad adicional (opcional): Para demostrar la efectividad de la protección CSRF, se puede usar `curl` para enviar un POST sin el token:

```
curl -X POST https://localhost:XXXX/Producto/Create \
-d "Nombre=Test&Precio=10&Stock=5&Categoria=Test" -k
```

El servidor responderá con HTTP 400 Bad Request porque `[ValidateAntiForgeryToken]` rechaza la petición al no encontrar el token válido en el formulario.

4.4. Caso 4 — Intento de inyección XSS en el campo Nombre

Datos de entrada: Nombre=<script>alert('xss')</script>, Descripción=Descripción normal, Precio=100, Stock=5, Categoría=Electrónica.

Resultado esperado: Los datos se aceptan (son sintácticamente válidos según las anotaciones), pero Razor **escapa automáticamente** el contenido al renderizarlo con `@p.Nombre`, impidiendo la ejecución del script.

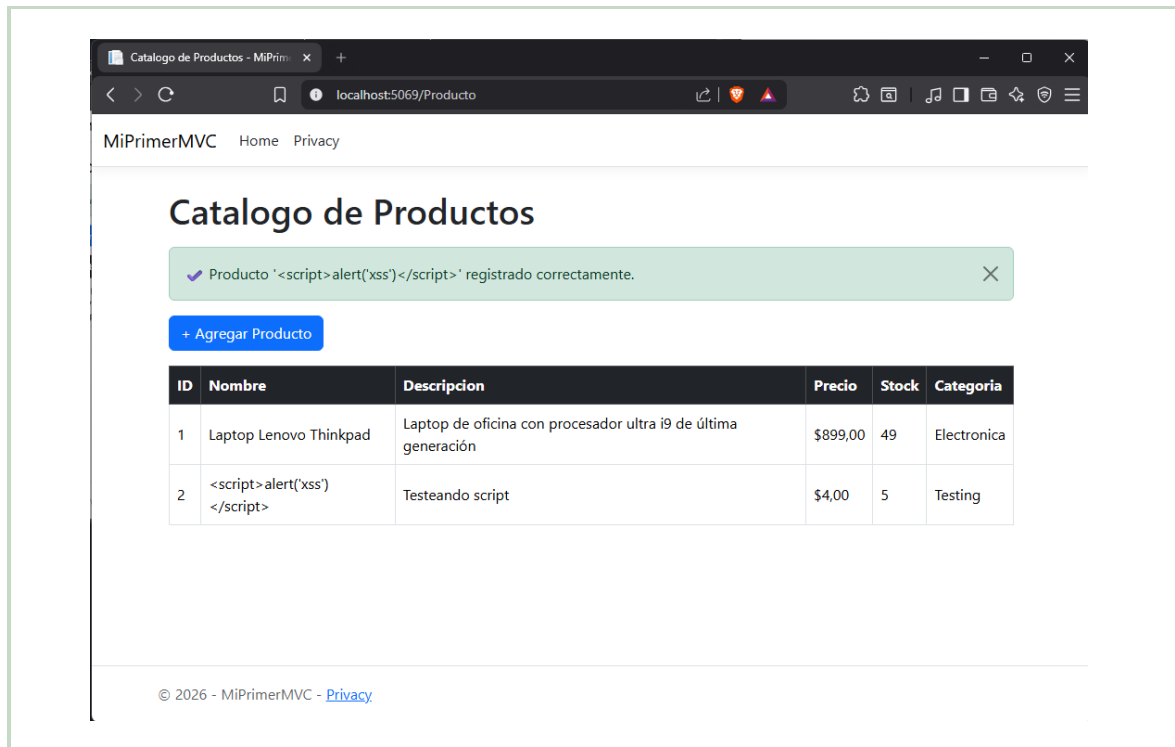


Figura 14: Página `/Producto/Index` mostrando en la tabla el nombre escapado como texto literal `<script>alert('xss')</script>` sin que se ejecute ningún alert, demostrando que el motor Razor aplica `HtmlEncoder.Default.Encode()` a toda expresión `@variable` por diseño.

Razor y el escape automático: A diferencia de PHP donde `echo $variable` produce salida sin sanear, en Razor la expresión `@variable` siempre llama a `HtmlEncoder.Default.Encode()` sobre el valor antes de escribirlo en la respuesta. Para producir HTML sin escapar explícitamente se requiere `@Html.Raw(variable)`, que debe usarse con extrema cautela [3].

4.5. Caso 5 — Datos completamente válidos y flujo exitoso

Datos de entrada: Nombre=*Laptop HP Pavilion 15*, Descripción=*Portatil 15.6 pulgadas, 16 GB RAM, 512 GB SSD*, Precio=*899.99*, Stock=*15*, Categoría=*Electrónica*.
Resultado esperado: `ModelState.IsValid = true`; el producto se agrega al repositorio, se aplica el patrón `Post/Redirect/Get` y se muestra el banner de éxito en la vista `Index`.

.NET 8 Formulario completado:

Registrar Nuevo Producto

Nombre del Producto

Laptop HP Pavilion 15

Descripcion

Laptop de última generación diseñada para satisfacer todas tus necesidades.

Precio (USD)

899.99

Stock disponible

34

Categoría

Electronica

Guardar Producto Cancelar

© 2026 - MiPrimerMVC - [Privacy](#)

Figura 15: Formulario `/Producto/Create` con todos los campos llenados con datos válidos: Nombre=Laptop HP Pavilion 15, Precio=899.99, Stock=15, Categoría=Electrónica.

.NET 8 Banner de éxito y tabla con productos:

Catalogo de Productos

✓ Producto 'Laptop HP Pavilion 15' registrado correctamente.

+ Agregar Producto

ID	Nombre	Descripcion	Precio	Stock	Categoría
1	Laptop Lenovo Thinkpad	Laptop de oficina con procesador ultra i9 de última generación	\$899,00	49	Electronica
2	<script>alert('xss')</script>	Testeando script	\$4,00	5	Testing
3	Laptop HP Pavilion 15	Laptop de última generación diseñada para satisfacer todas tus necesidades.	\$899,99	34	Electronica

© 2026 - MiPrimerMVC - [Privacy](#)

Figura 16: Página `/Producto/Index` tras el redirect exitoso, mostrando el banner verde con el mensaje “Producto ‘Laptop HP Pavilion 15’ registrado correctamente.” y la tabla con el producto recién agregado.

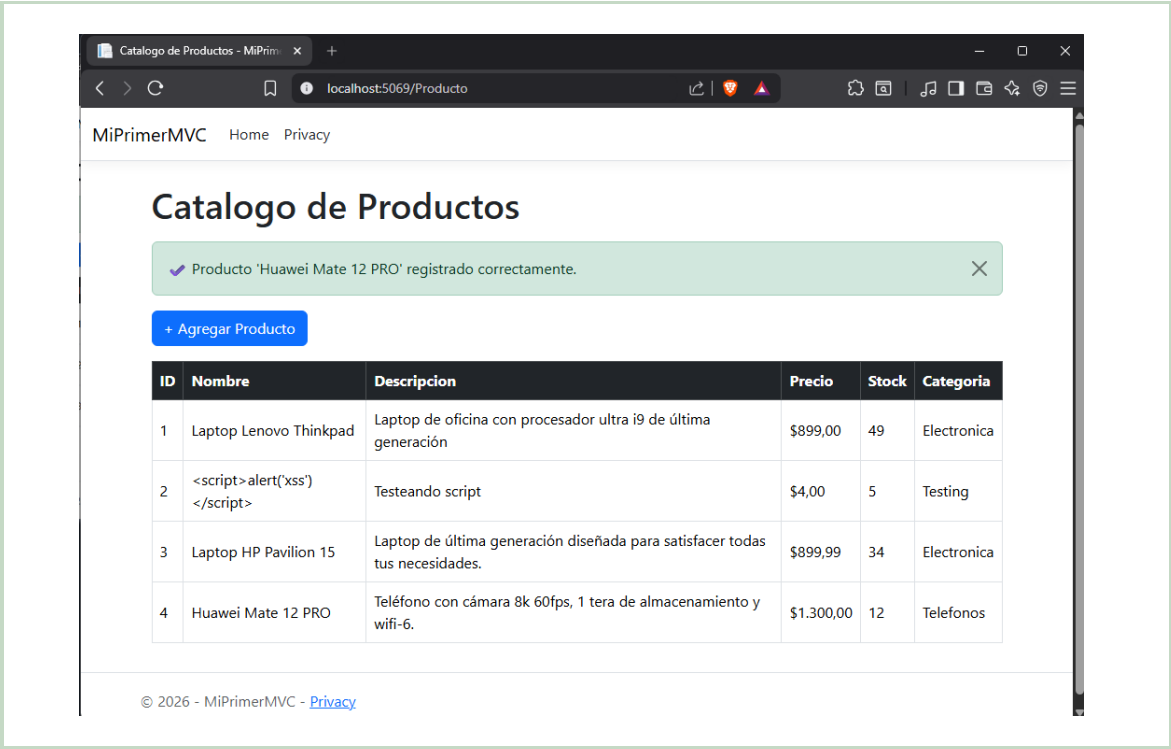


Figura 17: Página `/Producto/Index` con mínimo tres productos registrados de distintas categorías (Electrónica, Hogar, Deportes), evidenciando la acumulación correcta en el repositorio en memoria durante la sesión de la aplicación.

5. Conclusiones

Conclusión 1 — Separación de responsabilidades: El patrón MVC implementado en ASP.NET Core mantiene una separación limpia entre la lógica de dominio (`Producto.cs`), la lógica de flujo (`ProductoController`) y la presentación (`.cshtml`). Esta cohesión de cada capa y el acoplamiento mínimo entre ellas facilita el mantenimiento, las pruebas unitarias y la extensión futura (e.g., reemplazar el repositorio en memoria por Entity Framework Core sin tocar el controlador).

Conclusión 2 — Validación declarativa como contrato del modelo: Los *Data Annotations* permiten expresar las invariantes de negocio directamente en el modelo, convirtiendo la clase `Producto` en un *contrato ejecutable*: si los datos no cumplen los atributos, `ModelState.IsValid` garantiza que el cuerpo de la acción POST nunca recibirá un objeto inválido. Esta aproximación declarativa reduce el código de validación imperativo y centraliza las reglas en un solo lugar, minimizando la duplicación y los errores por omisión.

Conclusión 3 — Defensa en profundidad con doble capa de validación: La práctica confirmó el principio de *defensa en profundidad*: la validación en cliente (jQuery Validation) mejora la experiencia del usuario al proporcionar retroalimentación inmediata.

ción inmediata sin recargar la página, pero **nunca** puede sustituir a la validación en servidor, que es la única garantía real de integridad de los datos. Un atacante con conocimiento básico de HTTP puede eludir trivialmente la validación en cliente.

Conclusión 4 — Protección CSRF como capa obligatoria: La inspección del payload POST en DevTools confirmó que `asp-antiforgery="true"` inyecta un token criptográficamente firmado en cada formulario. El atributo `[ValidateAntiForgeryToken]` en la acción POST garantiza que solo las peticiones originadas desde el formulario legítimo del servidor sean aceptadas. Esta protección es obligatoria en cualquier formulario que modifique estado del servidor, según las guías de seguridad de OWASP [7].

Conclusión 5 — Escape automático como mitigación XSS: La prueba de inyección XSS demostró que el motor Razor escapa automáticamente toda expresión `@variable`, convirtiendo `<script>` en `<script>` antes de escribirla en la respuesta HTML. Este comportamiento seguro *por defecto* contrasta favorablemente con otros lenguajes de plantillas donde el escape es opt-in, reduciendo significativamente la superficie de ataque XSS en aplicaciones ASP.NET Core bien escritas.

Referencias

- [1] A. Lock, *ASP.NET Core in Action*, 3rd ed. Shelter Island, NY: Manning Publications, 2023, cubre ASP.NET Core 7 y 8; tratamiento exhaustivo del pipeline de middlewares, Razor Pages y MVC.
- [2] A. Freeman, *Pro ASP.NET Core 8: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages*, 9th ed. New York, NY: Apress, 2023, referencia exhaustiva de ASP.NET Core 8 con cobertura de Entity Framework Core, Identity y APIs REST.
- [3] Microsoft Corporation, “Overview of ASP.NET Core MVC,” Microsoft Learn, 2024, documentación oficial sobre el patrón MVC en ASP.NET Core. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview>
- [4] —, “Model validation in ASP.NET Core MVC and Razor Pages,” Microsoft Learn, 2024, referencia completa de Data Annotations y ModelState en el contexto de validación de formularios. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation>
- [5] —, “Model Binding in ASP.NET Core,” Microsoft Learn, 2024, explica la cadena de IValueProvider y el proceso de binding de tipos complejos. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/mvc/models/model-binding>
- [6] OWASP Foundation, “Input Validation Cheat Sheet,” OWASP Cheat Sheet Series, 2024, guía de mejores prácticas para validación de entradas en aplicaciones

web. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

- [7] —, “Cross-Site Request Forgery (CSRF),” OWASP Community Pages, 2023, descripción técnica del vector de ataque CSRF y mecanismos de mitigación recomendados. [Online]. Available: <https://owasp.org/www-community/attacks/csrf>
- [8] Microsoft Corporation, “Prevent Cross-Site Request Forgery (XSRF/CSRF) attacks in ASP.NET Core,” Microsoft Learn, 2024, documentación oficial del servicio IAntiforgery y del atributo ValidateAntiForgeryToken. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/security/anti-request-forgery>
- [9] —, “Tag Helpers in ASP.NET Core,” Microsoft Learn, 2024, describe los tag helpers incorporados incluyendo FormTagHelper, LabelTagHelper y ValidationMessageTagHelper. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/mvc/views/tag-helpers/intro>